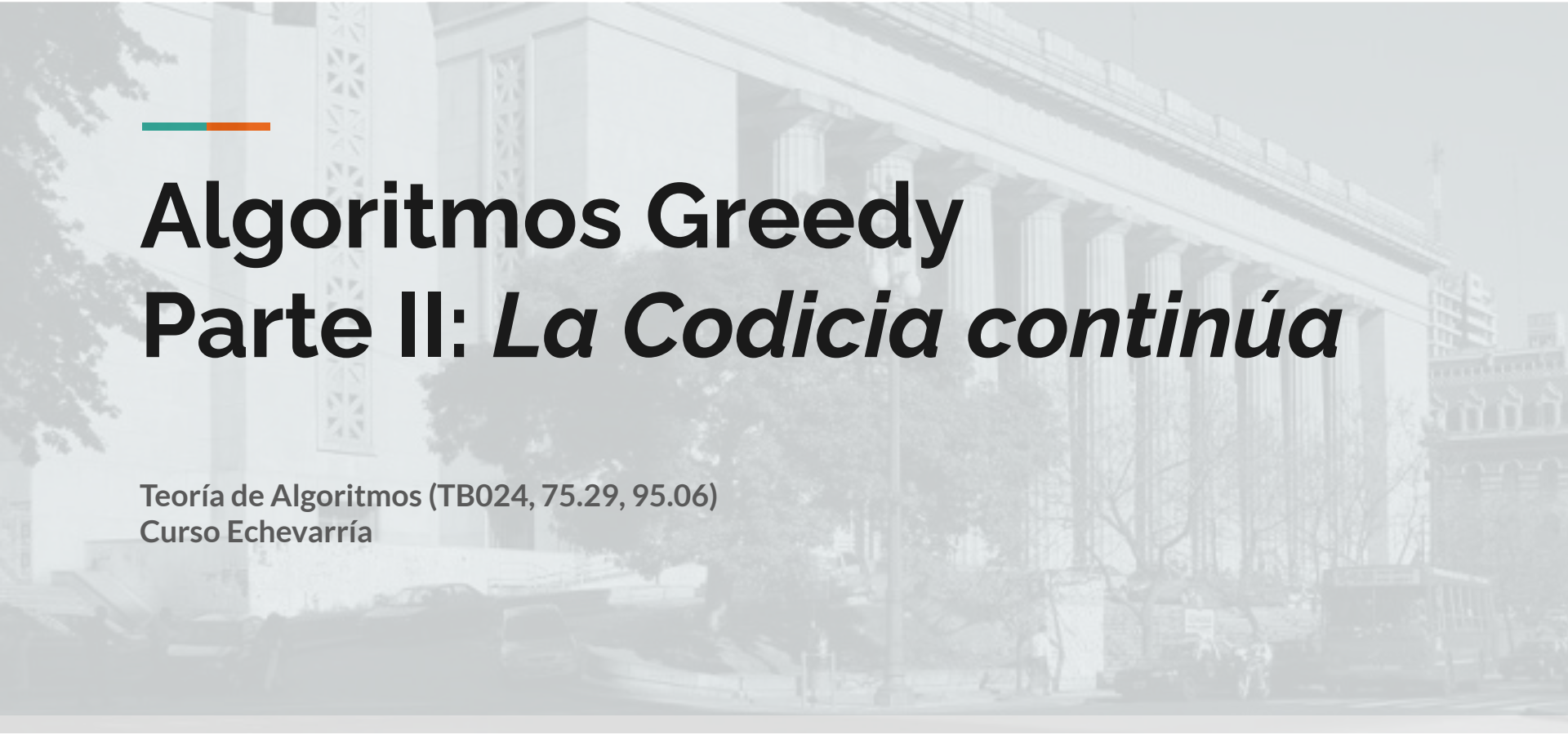


Algoritmos Greedy
Parte II: *La Codicia continúa*

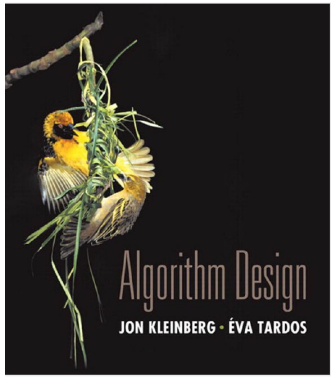
Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Repaso de la Clase Anterior

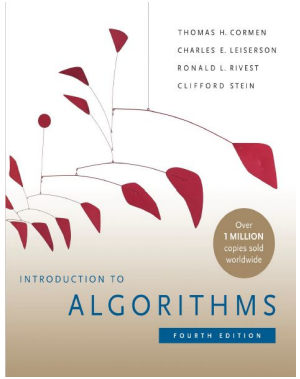


Repaso

- Dijkstra
- Prim y Kruskal
- Interval Scheduling
- Códigos de Huffman
- Compras con Inflación
- Carga de Combustible



Cap 4



Cap 15

Problema de la Mochila



Problema de la Mochila: Definición

- Se tiene una mochila con una capacidad máxima w
- Se tienen n objetos que ocupan una capacidad w_i y tienen un valor v_i
- Cada objeto puede incluirse en la mochila (si cabe) o ser excluido

Se desea determinar qué elementos poner en la mochila de forma tal que el valor total de los objetos incluidos sea máximo

Problema de la Mochila (cont.)

¿Posibles reglas greedy?

Problema de la Mochila (cont.)

Alternativa 1: agregar el próximo elemento de menor peso

1. Ordenar los objetos según su peso, de menor a mayor
2. Mientras quede capacidad, incorporar el próximo objeto

Objeto	Peso	Valor
a	1	7
b	3	4
c	4	8
d	6	14

Capacidad $w=5$
Solución: {a, b}
Valor alcanzado: 11

Pero la solución {a, c} es mejor

Problema de la Mochila (cont.)

Alternativa 2: agregar el próximo elemento de mayor valor

1. Ordenar los objetos según su valor, de mayor a menor
2. Mientras quede capacidad, incorporar el próximo objeto

Objeto	Peso	Valor
a	5	14
b	1	9
c	2	6
d	3	4

Capacidad $w=5$
Solución: {a}
Valor alcanzado: 14

Pero la solución {b, c} es mejor, con un valor de 15

Problema de la Mochila (cont.)

Alternativa 3: agregar el próximo elemento de mejor ponderación valor/peso

1. Ordenar los objetos según su relación valor/peso, de mayor a menor
2. Mientras quede capacidad, incorporar el próximo objeto

Objeto	Peso	Valor	Valor/Peso
a	1	3	3
b	5	14	2,80
c	2	4	2
d	3	5	1,66

Capacidad $w=5$
Solución: {a,c}
Valor alcanzado: 7

Pero la solución {b} es mejor

Problema de la Mochila

Jugando con los datos, puede forzarse al algoritmo a dar un resultado arbitrariamente malísimo:

Mochila con capacidad 100

w1: 1 p1: 2
w2: 100 p2: 100



Problema de la Mochila

Con un pequeño agregado, podemos obtener un algoritmo con garantía de calidad 1/2: La solución que me da nunca está por debajo de la mitad del óptimo.

1. Ordenar los objetos según su relación valor/peso, de mayor a menor
2. Mientras quede capacidad, incorporar el próximo objeto
    a. Cuando me encuentre con el primer objeto que no puedo sumar a la mochila, lo llamaré “elemento crítico” y lo guardo aparte.
3. Una vez completada la mochila, comparo el beneficio de la solución obtenida (P1) con el beneficio del elemento crítico (P2). Si es mayor, me quedo con la solución obtenida. Si no, armo una nueva solución que consista únicamente en el elemento crítico.

Problema de la Mochila

¿Por qué esto tiene garantía 1/2?

- El elemento crítico es el primero que queda afuera, entonces sabemos que: $W1<W$, $W2<W$ y $W1+W2>W$
- Los elementos están ordenados por el coeficiente en forma descendente, entonces: $P1/W1 > P2/W2$

Entonces, sabemos que:

- Los elementos de P1+P2 son los que mejor aprovechan el espacio disponible.
- Por lo menos una de las dos soluciones (W1,W2) ocupa más de la mitad de la mochila.

  ⇒ La solución obtenida es por lo menos la mitad de la solución óptima

Problema de la Mochila

No se conoce un algoritmo greedy que siempre encuentre la solución óptima al problema de la mochila.

¿Por qué es tan difícil?

- *Tengo n posibles elementos.*
- *Cada elemento puede estar o no estar*

¿Cuántas formas de combinarlos hay?

- *Por fuerza bruta, debería generar todas esas combinaciones, descartar las incompatible, y buscar la mayor del resto.*

Variante: Objetos Fraccionables

Tomemos el problema de la mochila anterior:

- ¿Cómo cambia si ahora podemos incluir una fracción de cada objeto?

¿Podemos encontrar un algoritmo greedy que SIEMPRE encuentre el óptimo?

Scheduling con Minimización de Retrasos



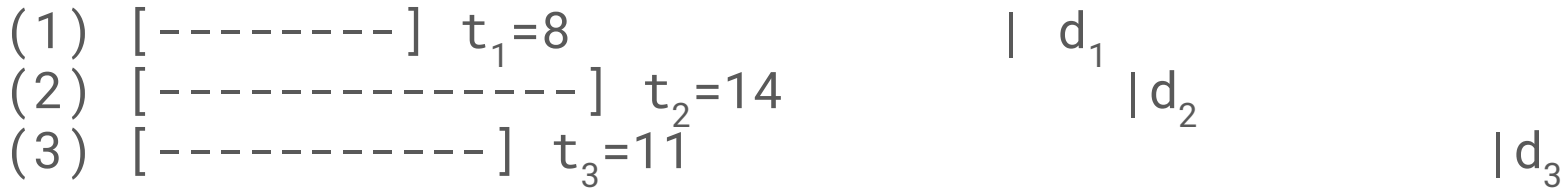
Scheduling con Minimización de Retrasos

- Tenemos un conjunto de tareas $S=\{1,2,\ldots, n\}$
- La $i\text{-}ésima$ tarea tiene un deadline d_i y una duración t_i
- Pueden empezar en cualquier momento, pero no pueden ser interrumpidas
- No puede haber solapamiento entre las tareas
- Si termina después de su deadline, hay una penalización
- **Todas** las tareas deben ser ejecutadas

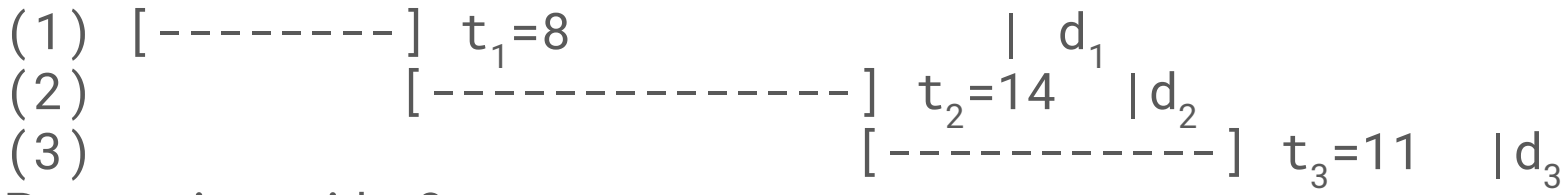
Queremos encontrar el secuenciamiento de tareas que minimice el retraso total

Scheduling con Retrasos (cont.)

Ejemplo:



Solución:



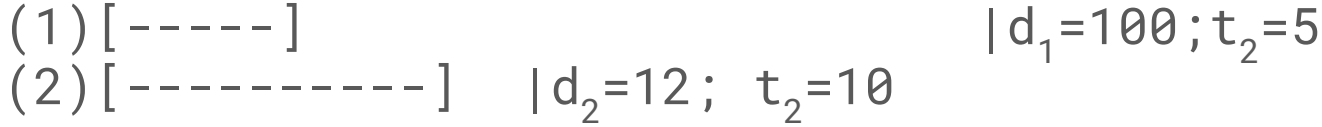
Demora incurrida: 0

Scheduling con Retrasos (cont.)

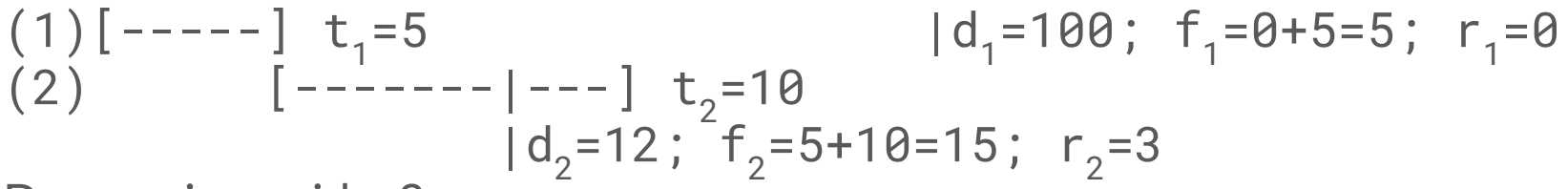
¿Posibles reglas greedy?

Scheduling con Retrasos (cont.)

Opción 1: Tomar la tarea de menor duración



Solución:

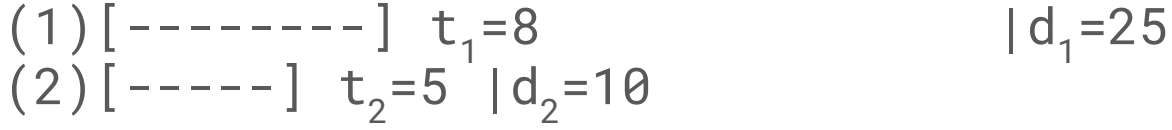


Demora incurrida: 3

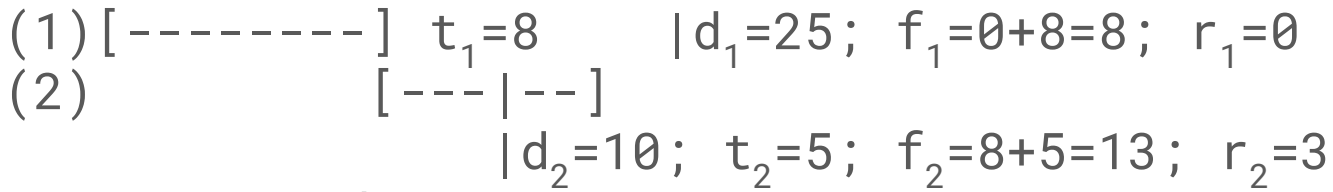
Pero se podrían haber calendarizado al revés, y no hubiera habido retraso

Scheduling con Retrasos (cont.)

Opción 2: Tomar la tarea de mayor duración



Solución:



Demora Incurrida: 3

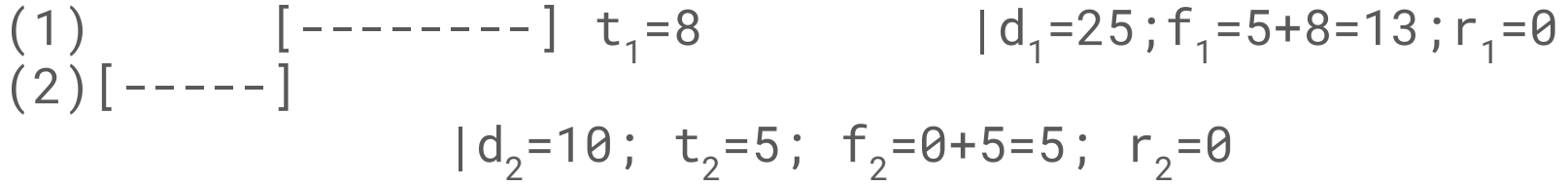
Pero se podrían haber calendarizado al revés, y no hubiera habido retraso

Scheduling con Retrasos (cont.)

Opción 3: Tomar la tarea con deadline más próximo



Solución:

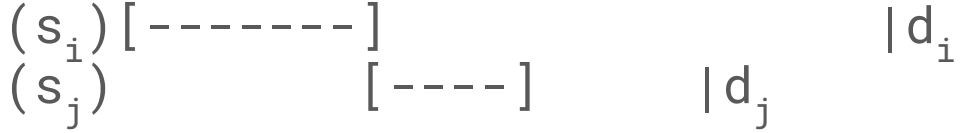


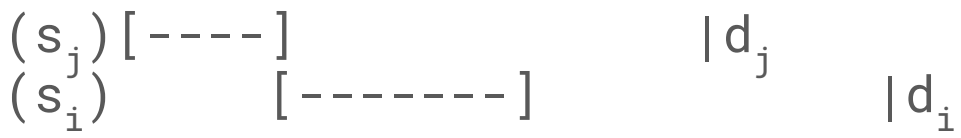
Esta opción *siempre* da una solución óptima, aún sin tener en cuenta las duraciones de las tareas

Scheduling con Retrasos: Demostración

Definición de Inversión:

Un schedule tiene inversiones si existen dos tareas s_i, s_j tales que $i<j$ y $d_i>d_j$ (es decir, si la tarea s_i se programó antes que s_j, pero s_j tiene deadline anterior a s_i)





Scheduling con Retrasos: Demostración

Hipótesis: Sean S1 y S2 dos schedules posibles sin inversiones

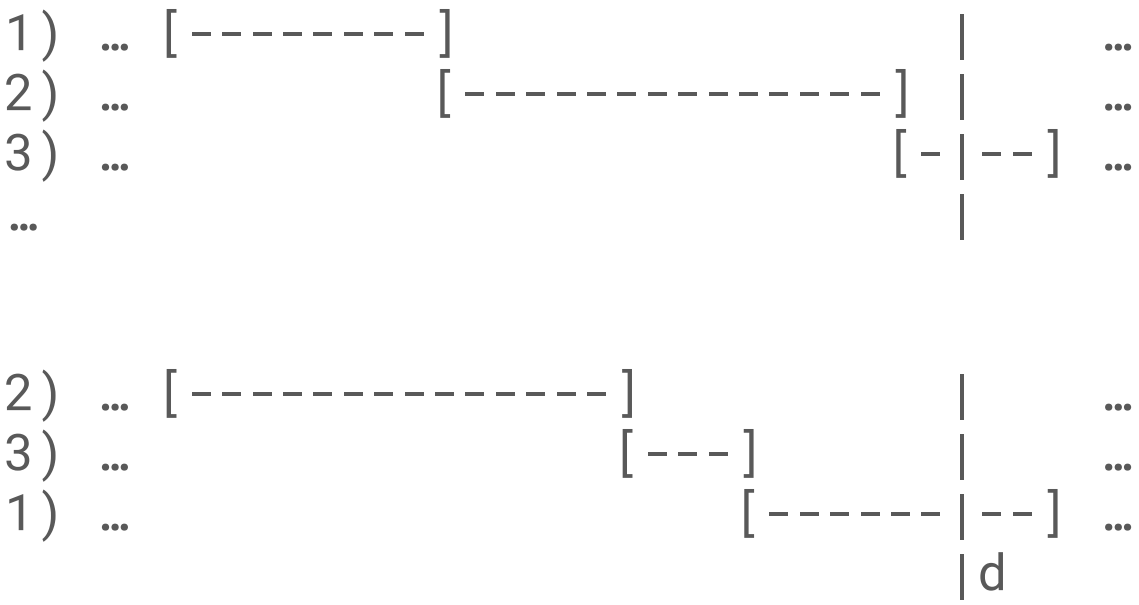
Tesis: S1 y S2 tienen la misma demora máxima

Demostración:

Como no tienen inversiones, la única forma de que sean distintos es que haya tareas con el mismo deadline pero secuenciadas en distinto orden, y tienen que ser consecutivas.

La última de estas tareas es la que determina la demora máxima del schedule, y no depende del orden de las tareas.

Scheduling con Retrasos: Demostración



Scheduling con Retrasos: Demostración

Hipótesis: Sea S un schedule sin inversiones y sin tiempo ocioso entre tareas

Tesis: S es óptimo

Demostración

Supongamos que existe un schedule S’ que es óptimo pero que tiene inversiones. Si tiene inversiones, podría cambiar el orden de las tareas invertidas (que son contiguas). Al invertir las tareas inversibles la solución S’ no puede mejorar (porque ya es óptima) pero tampoco puede empeorar (porque el retraso tampoco cambia): debe quedar igual.

Scheduling con Retrasos: Demostración

1. Dos schedules S y S’ sin inversiones tienen el mismo valor de demora
2. Un schedule S sin inversiones es óptimo

Corolario:

- Todo schedule S sin inversiones es óptimo

Como nuestro algoritmo genera un schedule sin inversiones, nuestro algoritmo entrega soluciones óptimas

Coloreo o Particionamiento de Intervalos



Coloreo de Intervalos: Definición

- Tenemos un conjunto de tareas $S=\{1,2,\ldots, n\}$
- La $i\text{-}ésima$ tarea empieza en el horario $s(i)$ y termina en el horario $f(i)$
- Tenemos k procesadores para asignarles tareas
- Un mismo procesador no puede ejecutar dos tareas a la vez
- Se deben asignar **todas** las tareas
- Cuidado: si k es demasiado pequeño, puede no existir solución

Encontrar (si existe) la asignación de tareas a procesadores que haga mínima la cantidad de procesadores utilizados

Coloreo de Intervalos: Definición (cont.)

Ejemplo (k=2):

```
1) [1111111111]
2)                                  [22222]
3)          [333333333333333]
4)               [444444444]
```

Solución:

```
A) [1111111111][444444444]
B)          [333333333333333][22222]
```

Procesador A: tareas 1 y 4
Procesador B: tareas 2 y 3

Coloreo de Intervalos (cont.)

Planteamos un algoritmo greedy para resolverlo:

1. Seleccionar una tarea i_1 siguiendo una regla sencilla
2. Si es posible:
    a. Asignarla a un procesador libre previamente utilizado
3. Si no:
    a. Si quedan procesadores disponibles:
        i. Asignarla a un nuevo procesador
    b. Si no:
        i. El problema no tiene solución
4. Repetir hasta que no queden tareas

Coloreo de Intervalos (cont.)

¿Posibles reglas greedy?

Coloreo de Intervalos (cont.)

Opción 1 (k=2): tomar la tarea que *termine* antes

```
1) [1111]
2) [222222]
3)          [3333]
4)                  [4444]
5)      [5555555555555]
```

Con este algoritmo no hay solución (no se puede ubicar la tarea 5):

```
A) [1111]    [3333] [4444]
B) [222222]
```

Pero sí existe una solución:

```
A) [1111] [5555555555555]
B) [2222222] [3333] [4444]
```

Coloreo de Intervalos (cont.)

Opción 2 (k=2): tomar la tarea que *empiece* antes

```
1) [1111]
2) [222222]
3)          [3333]
4)                  [4444]
5)      [5555555555555]
```

Solución:

```
A) [1111] [5555555555555]
B) [2222222] [3333] [4444]
```

Si el problema tiene solución, esta variante siempre encuentra la óptima

Optimal Caching



Optimal Caching: Definición

- Tenemos un conjunto U de n elementos en la memoria principal
- Tenemos también una memoria cache C, más rápida, que puede contener $k<n$ elementos
- C tiene inicialmente k elementos ya cargados
- Se desea leer una secuencia $D=d_1, d_2, \ldots, d_m$ de elementos desde U.
- Debemos decidir en todo momento qué k elementos mantener en C para un acceso más rápido
- Si el elemento d_i no está en C (cache miss), debe ser buscado en U, incurriendo en una penalización, y debe removerse (evict) un elemento de C para dar lugar a d_i

Se desea minimizar la cantidad de misses

Optimal Caching: Definición (cont.)

El algoritmo de mantenimiento del cache determina un *eviction schedule*, estableciendo qué elementos deben estar C en todo momento.

Optimal Caching: Ejemplo

- Supongamos que:
  - $k=2$
  - $U=\{a, b, c\}$
  - $D=\{a, b, c, b, c, a, b\}$
  - $C_o=\{a, b\}$

C	*Elemento a Leer*	*Hit/Miss*	*Evicted Element*
{a,b}	a	Hit	
{a,b}	b	Hit	
{a,b}	c	Miss	a
{c,b}	b	Hit	
{c,b}	c	Hit	
{c,b}	a	Miss	c
{a,b}	b	Hit	

Optimal Caching: La regla Greedy

Farthest in the Future

Remover de C el elemento que se va a usar lo más adelante posible

Se demuestra que es la mejor elección, y es óptima (*)



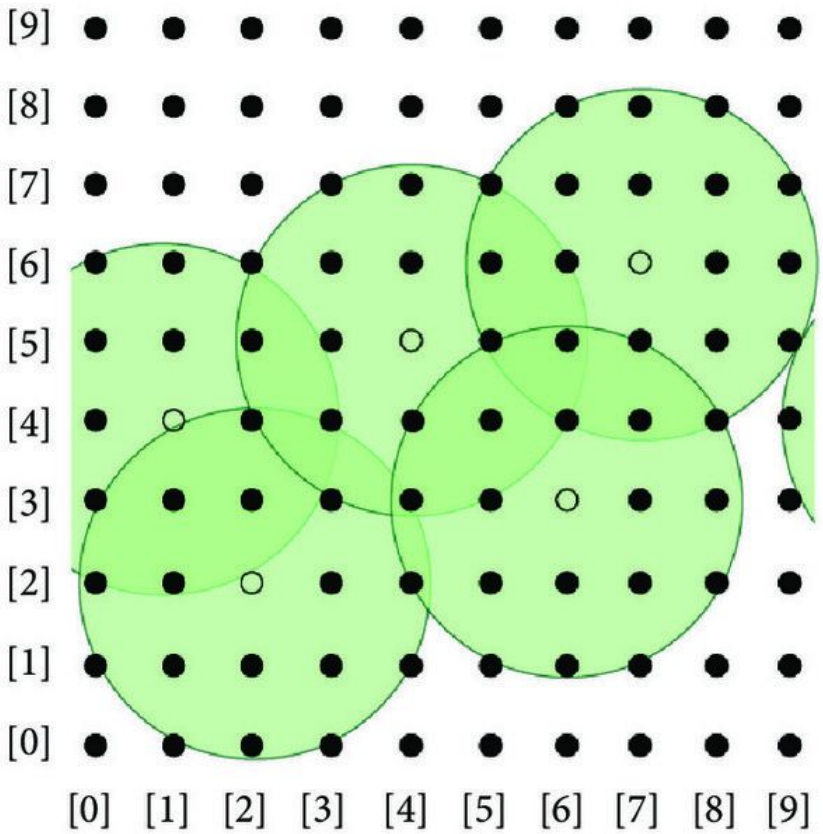
(*) Esto es cierto si se conoce de antemano la secuencia D

Optimal Caching: La vida real

En la vida real no contamos con una bola de cristal para predecir la secuencia D.

En la práctica, la mejor elección es usar un algoritmo LRU: Least Recently Used. Se resume en remover de C el elemento cuyo último acceso ocurrió hace más tiempo.

Set Covering (Cobertura de conjuntos)



Set Covering: Definición

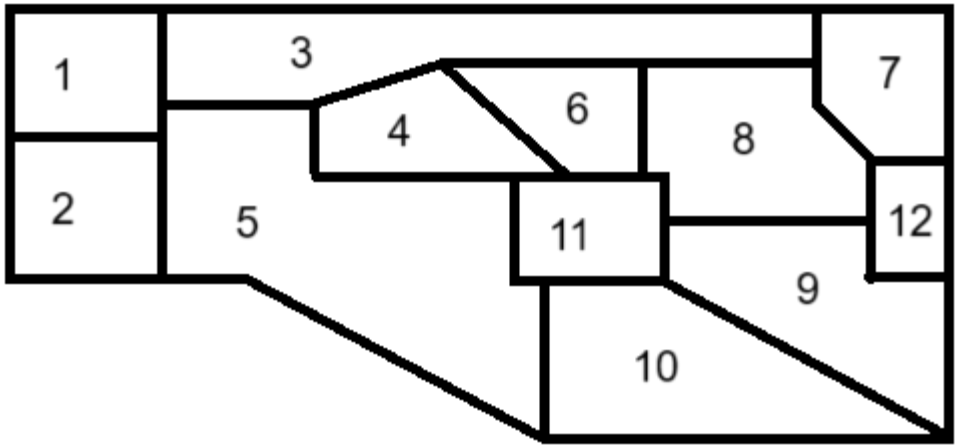
- Se tiene un conjunto de m elementos $U=\{u_1, u_2, \ldots, u_m\}$ (universo)
- También se tienen n conjuntos de dichos elementos, $S=\{S_1, S_2, \ldots, S_n\}$, cuya unión es U

Determinar el menor número de conjuntos de S cuya unión aún tiene todos los elementos de U

Set Covering: Ejemplo

Una estación de bomberos tiene la capacidad de cubrir las emergencias tanto de su comuna como de las comunas adyacentes a ella. Se necesitan construir tantas estaciones de bomberos como sea necesario para cubrir todas las comunas ante posibles emergencias, cuidando que todas las comunas estén cubiertas por al menos una estación (una o más), minimizando el número de estaciones construidas

Set Covering: Ejemplo (cont.)



Set Covering: Ejemplo (cont.)

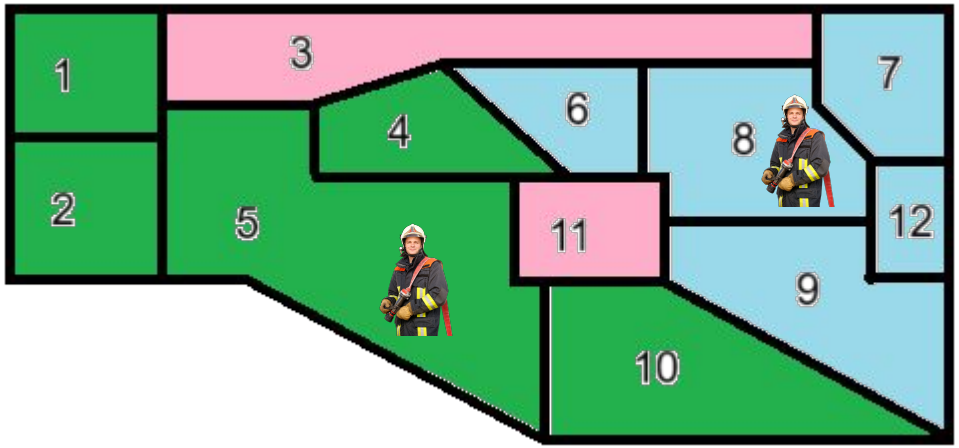
Estación en 5; cubre:

- 1, 2, 3, 4, 5, 10 y 11

Estación en 8; cubre:

- 3, 6, 7, 8, 9, 11 y 12

Las comunas 3 y 11 son cubertas por las dos estaciones



Set Covering (cont.)

Planteamos un algoritmo greedy para resolverlo:

1. Seleccionar un conjunto S_i siguiendo una regla sencilla
2. Si quedan elementos por incluir:
    a. Remover S_i de S
    b. Continuar
3. Si no:
    a. Terminar

Set Covering (cont.)

¿Posibles reglas greedy?

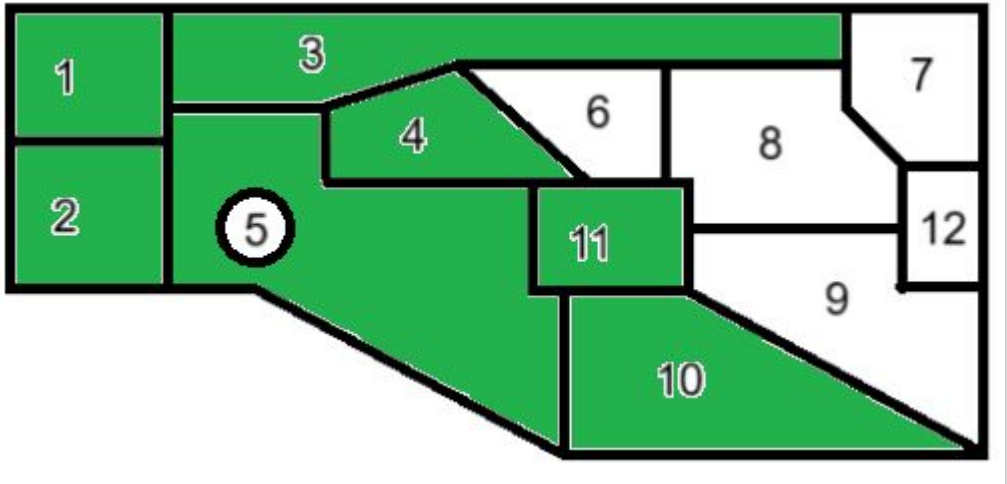
Set Covering (cont.)

Regla Greedy: Seleccionar el conjunto S_i que tenga la mayor cantidad de elementos aún no incluidos

- {1,2,3,5}
- {2,1,5}
- {3,1,5,4,6,8,7}
- {4,3,5,11,6}
- {5,1,2,3,4,10,11}
- {6,3,4,8,11}
- {7,3,8,12}
- {8,3,6,7,9,11,12}
- {9,8,10,11,12}
- {10,5,9,11}
- {11,4,5,6,8,9,10}
- {12,7,8,9}

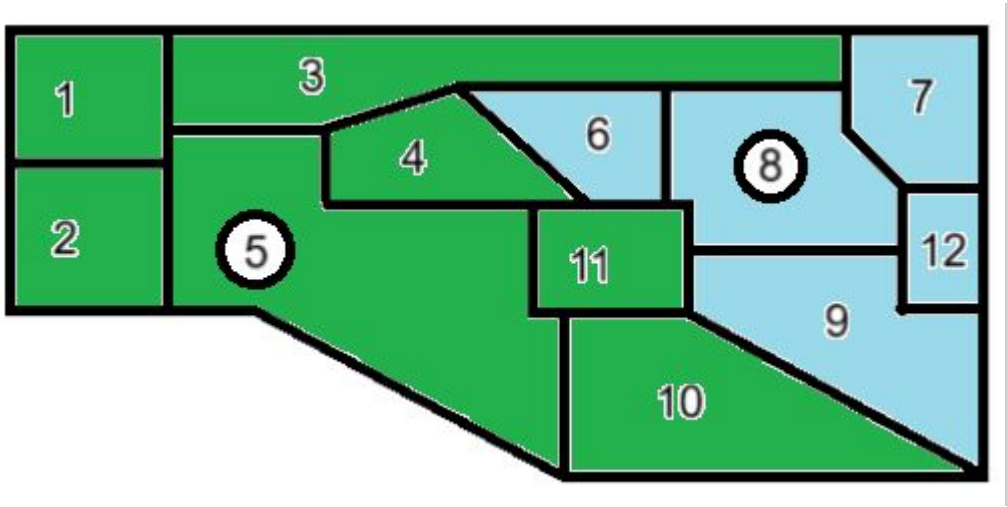
Set Covering (cont.)

- {5,1,2,3,4,10,11}
- {11,4,5,6,8,9,10}
- {8,3,6,7,9,11,12}
- {3,1,5,4,6,8,7}
- {4,3,5,11,6}
- {6,3,4,8,11}
- {9,8,10,11,12}
- {1,2,3,5}
- {7,3,8,12}
- {10,5,9,11}
- {12,7,8,9}
- {2,1,5}



Set Covering (cont.)

- {5,1,2,3,4,10,11}
- {8,~~3~~,6,7,9,~~11~~,12}
- {~~11~~,~~4~~,~~5~~,6,8,9,~~10~~}
- {~~3~~,~~1~~,~~5~~,~~4~~,6,8,7}
- {~~4~~,~~3~~,~~5~~,~~11~~,6}
- {6,~~3~~,~~4~~,8,~~11~~}
- {9,8,~~10~~,~~11~~,12}
- {~~1~~,~~2~~,~~3~~,~~5~~}
- {7,~~3~~,8,12}
- {~~10~~,~~5~~,9,~~11~~}
- {12,7,8,9}
- {~~2~~,~~1~~,~~5~~}



Problema del Viajante de Comercio



Viajante de Comercio: Definición

- Se tiene una lista de ciudades y las distancias entre ellas

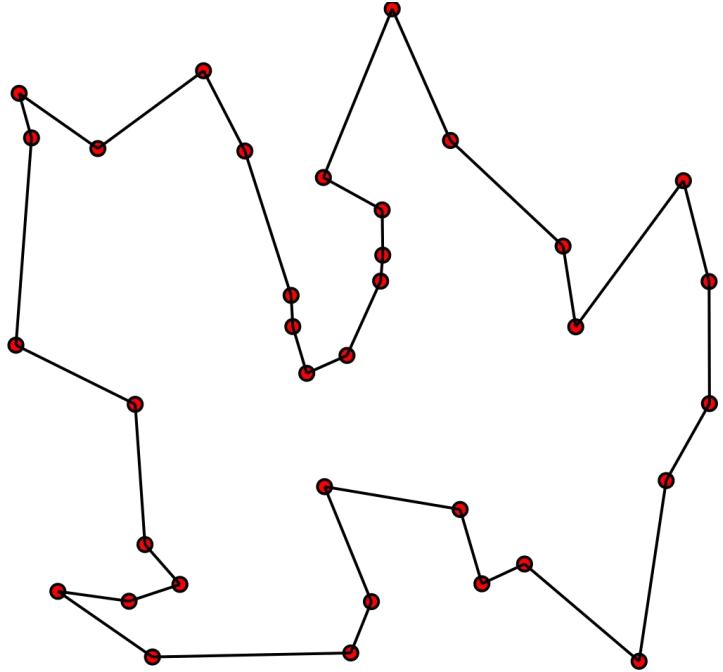
¿Cuál es la ruta más corta que, partiendo de una de ellas y regresando a la misma, visita todas las ciudades exactamente una vez?

Viajante de Comercio: Definición Alternativa

- Sea G un grafo ponderado $G=(V, E)$ completo
- G puede ser dirigido o no dirigido

Encontrar el ciclo Hamiltoniano de menor peso

Viajante de Comercio: Ejemplo



Viajante de Comercio: Características

- La solución exacta es muy difícil (costosa) de encontrar: hay $n!$ combinaciones para explorar
- Su formulación tan simple y su gran dificultad para ser resuelto lo hace muy popular
- Tiene muchísimas aplicaciones:
  - Logística y planificación
  - Diseño de soldaduras
  - Manufactura de microchips
  - Secuenciamiento de cadenas de ADN
  - Astronomía: cálculo del mínimo movimiento para apuntar un telescopio
  - El GPS
  - ...

Viajante de Comercio: Características

- Hay muchos tipos de algoritmos para encontrar soluciones (normalmente sub-óptimas)
  - Greedy
  - Búsqueda Tabú
  - Simulated Annealing
  - Colonias de Hormigas
  - Algoritmos Genéticos
  - ...



Viajante de Comercio (cont.)

¿Posibles reglas greedy?

Viajante de Comercio (cont.)

Regla greedy: seleccionar el vecino más cercano

1. Ordenar la lista de tramos por distancia, de menor a mayor
2. Tomar la ciudad origen
3. Buscar el tramo más corto que parta de la ciudad actual y que no cierre el ciclo
4. Acumular la distancia recorrida por el tramo seleccionado
5. Remover los tramos que salen de la ciudad actual
6. Remover los tramos que llegan a la siguiente ciudad
7. Avanzar hasta la nueva ciudad
8. Repetir desde 3 hasta que no queden ciudades por visitar
9. Sumar la distancia desde la última ciudad visitada hacia el origen (*)

(*) Este paso es necesario para cerrar el ciclo al terminar

Viajante de Comercio (cont.)

El recorrido empieza y termina en A

Lista inicial ordenada por distancia de menor a mayor:

- {ad,1}
- {da,1}
- {ac,2}
- {bc,2}
- {bd,2}
- {ca,2}
- {cb,2}
- {ce,2}
- {db,2}
- {de,2}
- {ec,2}
- {ed,2}
- {ab,3}
- {ba,3}
- {cd,3}
- {dc,3}
- {be,4}
- {eb,4}
- {ae,5}
- {ea,5}

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Viajante de Comercio (cont.)

Paso 1: tomamos {ad,1}

Recorrido: {ad,1}

- {ad,1}
- {da,1}
- ~~{ac,2}~~
- {bc,2}
- ~~{bd,2}~~
- {ca,2}
- {cb,2}
- {ce,2}
- {db,2}
- {de,2}
- {ec,2}
- ~~{ed,2}~~
- ~~{ab,3}~~
- {ba,3}
- ~~{cd,3}~~
- {dc,3}
- {be,4}
- {eb,4}
- ~~{ae,5}~~
- {ea,5}

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Viajante de Comercio (cont.)

Paso 2: rechazamos {da,1} porque cierra un ciclo

Tomamos {db,2}

Recorrido: {ad,1}{db,2}

- {ad,1}
- ~~{da,1}~~
- ~~{ac,2}~~
- {bc,2}
- ~~{bd,2}~~
- {ca,2}
- ~~{cb,2}~~
- {ce,2}

- {db,2}
- ~~{dc,2}~~
- {ec,2}
- ~~{ed,2}~~
- ~~{ab,3}~~
- {ba,3}

- ~~{cd,3}~~
- ~~{dc,3}~~
- {be,4}
- ~~{eb,4}~~
- ~~{ae,5}~~
- {ea,5}

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Viajante de Comercio (cont.)

Paso 3: tomamos {bc,2}

Recorrido: {ad,1}{db,2}{bc,2}

- {ad,1}
- ~~{da,1}~~
- ~~{ac,2}~~
- {bc,2}
- ~~{bd,2}~~
- {ca,2}
- ~~{cb,2}~~
- {ce,2}

- {db,2}
- ~~{de,2}~~
- ~~{ec,2}~~
- ~~{ed,2}~~
- ~~{ab,3}~~
- ~~{ba,3}~~

- ~~{cd,3}~~
- ~~{dc,3}~~
- ~~{be,4}~~
- ~~{eb,4}~~
- ~~{ae,5}~~
- {ea,5}

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Viajante de Comercio (cont.)

Paso 4: rechazamos {ca,2} porque forma un ciclo; tomamos {ce,2}

Recorrido: {ad,1}{db,2}{bc,2}{ce,2}

- {ad,1}
- ~~{da,1}~~
- ~~{ac,2}~~
- {bc,2}
- ~~{bd,2}~~
- ~~{ca,2}~~
- ~~{cb,2}~~
- {ce,2}

- {db,2}
- ~~{de,2}~~
- ~~{ec,2}~~
- ~~{ed,2}~~
- ~~{ab,3}~~
- ~~{ba,3}~~

- ~~{cd,3}~~
- ~~{dc,3}~~
- ~~{be,4}~~
- ~~{eb,4}~~
- ~~{ae,5}~~
- {ea,5}

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Viajante de Comercio (cont.)

Paso Final: se visitaron todos las ciudades y cerramos el ciclo con {ea,5}

Recorrido: {ad,1}{db,2}{bc,2}{ce,2}{ea,5}

Distancia: 12

- {ad,1}
- ~~{da,1}~~
- ~~{ac,2}~~
- {bc,2}
- ~~{bd,2}~~
- ~~{ca,2}~~
- ~~{cb,2}~~
- {ce,2}

- {db,2}
- ~~{dc,2}~~
- ~~{cc,2}~~
- ~~{ed,2}~~
- ~~{ab,3}~~
- ~~{ba,3}~~

- ~~{cd,3}~~
- ~~{dc,3}~~
- ~~{bc,4}~~
- ~~{eb,4}~~
- ~~{ae,5}~~
- {ea,5}

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Viajante de Comercio (cont.)

¿Qué habría pasado si hubiéramos hecho {ad,1}{de,2}{ec,2}{cb,2}{ba,3}?

La distancia habría sido 10

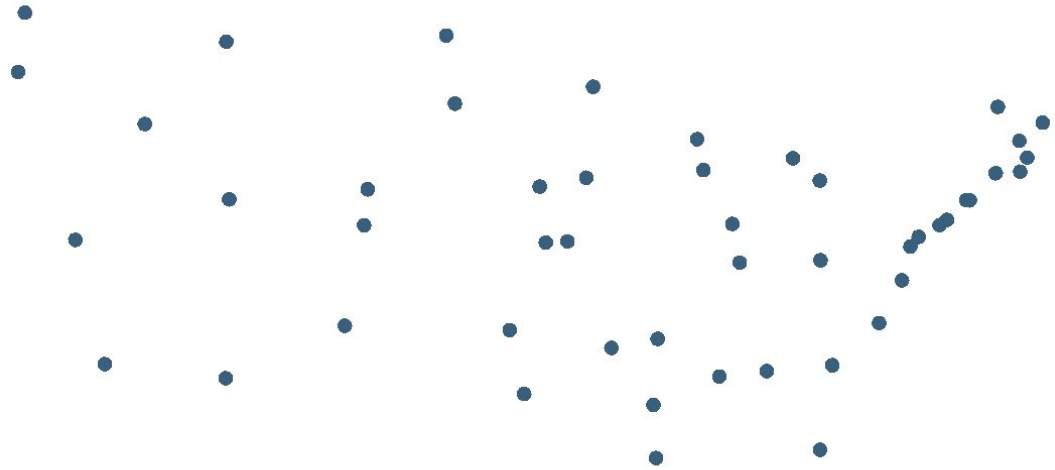
	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

La debilidad de este algoritmo en particular es que puede dejar las aristas más pesadas para el final

Viajante de Comercio (cont.)

- Vecino más cercano:

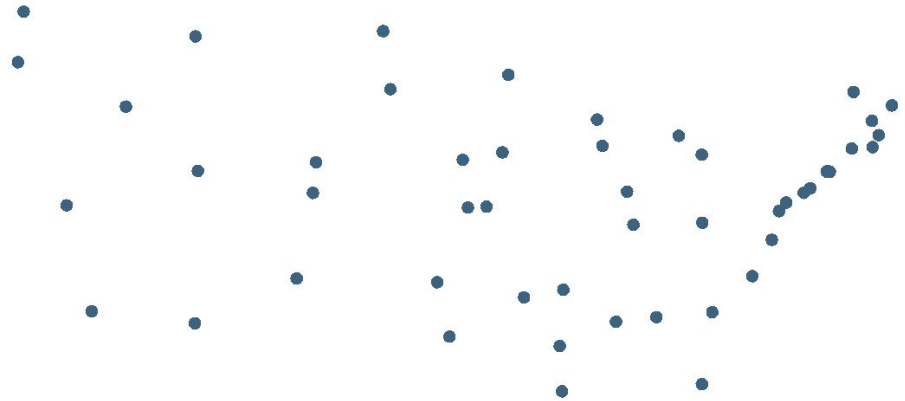
La debilidad de este algoritmo en particular es que puede dejar las aristas más pesadas para el final



Viajante de Comercio (cont.)

- Tramo más corto:

1. Ordenar los ejes de menor a mayor
2. Tomar el primer eje de la lista, siempre que sea “legal”:
    1. No cierre un ciclo con menos de N ejes
    2. No eleve el grado de ningún nodo a más de 2
3. Si el tour tiene menos de N nodos, volver a 2





¡Muchas gracias!